



Tabaani

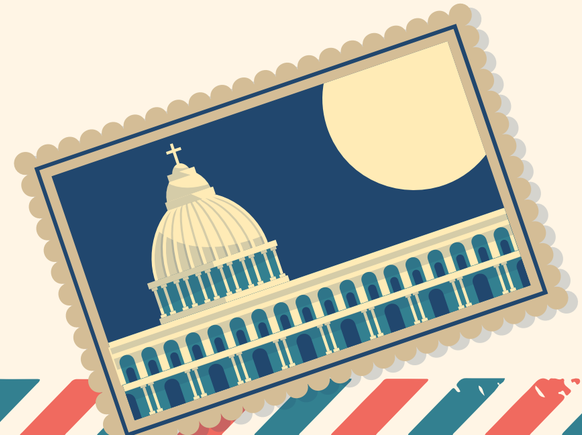
Plateforme collaborative de roadtrips et de sorties pédagogiques

Architecture Web Distribuée

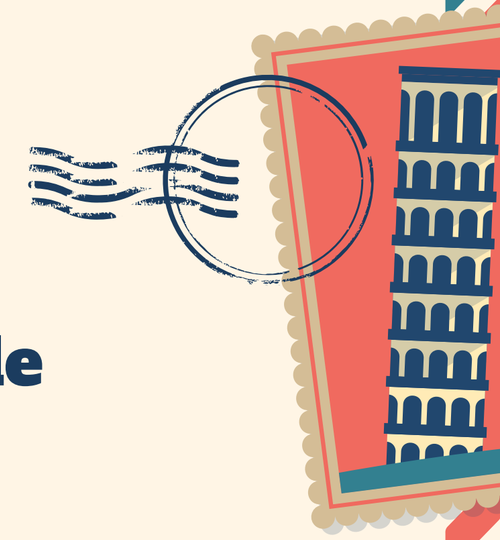
4SSA13 - Groupe 3

2026-2027





Plan :



01 Présentation générale du projet

02 Analyse fonctionnelle

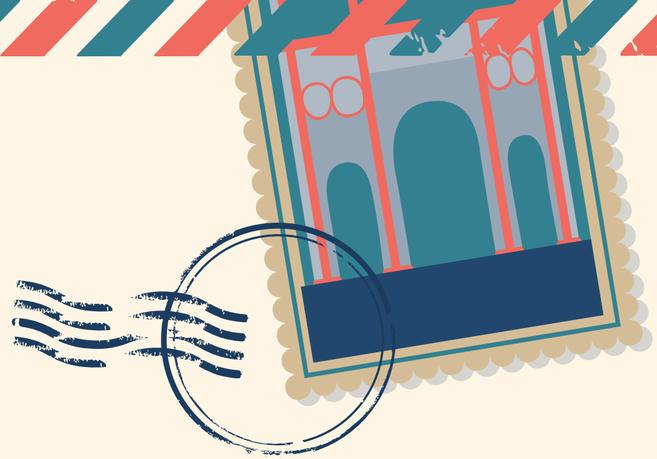
03 Architecture globale et technologies

04 Conception des microservices

05 Organisation et gestion du projet

06 Planification et perspectives





01
Présentation générale du projet





Idée générale et problème

- **Idée :**

organiser un roadtrip ou une sortie pédagogique, au même endroit

- **Problème :**

infos éparpillées, pas de suivi, frais compliqués



Problématique et cible

- **Problématique :**

« Comment aider les voyageurs, surtout les étudiants, à organiser et vivre un voyage simplement ? »

- **Cible:**

accès pour tous, avec des forfaits et avantages préférentiels pour les étudiants.



Ce qui rend Tabaani différent



Tout-en-un



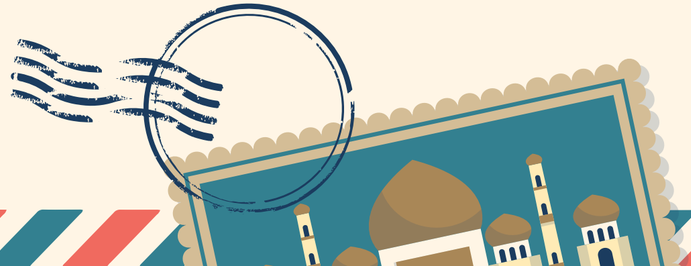
Ouvert à tous



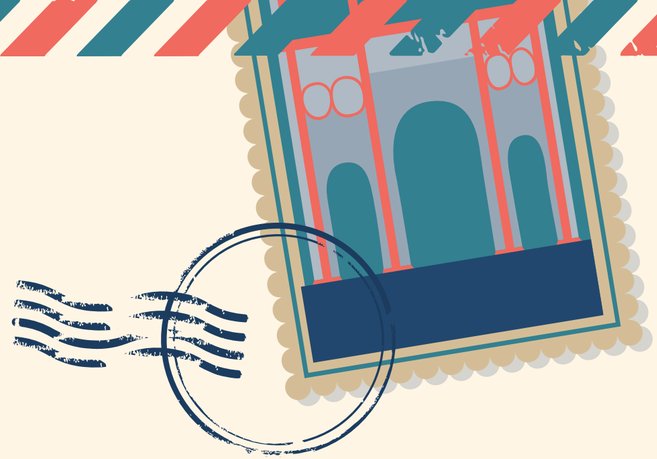
Forfaits étudiants



Suivi du groupe en direct







02
Analyse fonctionnelle





Acteurs du système



Voyageurs

Découvrent des lieux, participent aux voyages, réservent et partagent les frais.



Organisateurs

Créent les voyages, invitent les participants et coordonnent les étapes et les dates.



Prestataires

Les vendeurs proposent du matériel à acheter ou à louer. Les propriétaires proposent des voitures et gèrent leurs disponibilités.

Les étudiants vérifiés bénéficient d'offres préférentielles.

Une personne peut cumuler plusieurs rôles.



Principales fonctionnalités



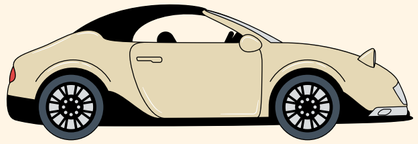
Planifier ensemble

Créer un voyage, inviter les membres et voter sur les étapes et les dates.



Découvrir des lieux

Explorer des lieux, rechercher à proximité et consulter les photos et les avis.



Réserver une voiture

Consulter les disponibilités, réserver un véhicule et partager les places.



S’équiper

Acheter ou louer du matériel pour préparer le voyage.



Partager les frais

Effectuer les paiements et répartir les dépenses entre les participants.

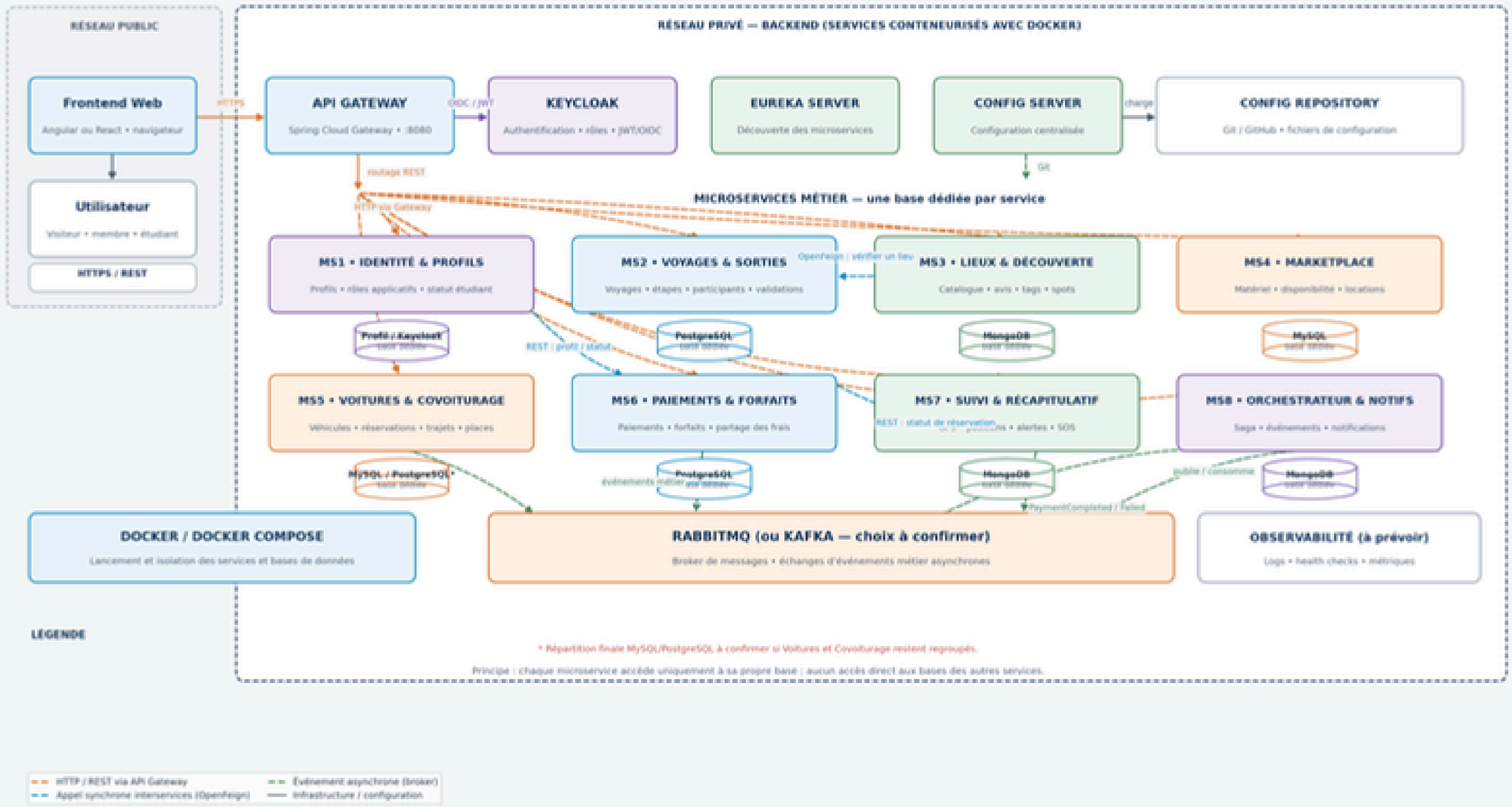


Suivre le voyage

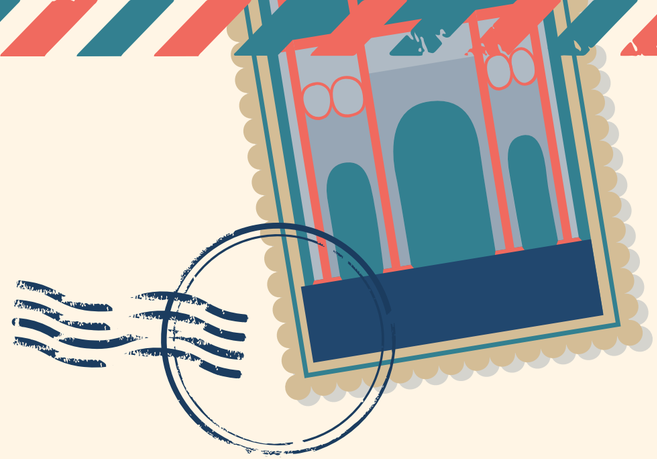
Localiser le convoi en direct et retrouver un récapitulatif du voyage.

VUE D'ENSEMBLE DE L'ARCHITECTURE MICROSERVICES — TABAANI

Roadtrips • sorties pédagogiques • mobilité • services aux voyageurs







03
Architecture globale et technologies





Architecture globale

Diagramme d’architecture

Réseau public

application web (Angular / React)

API Gateway

8080, routes /api/trips/** → lb://VOYAGES

Keycloak

8180, realm tabaani

Eureka

8761 · Config Server :8888 ← config-repo (GitHub)

Réseau privé

7 microservices + 1 commun, un conteneur chacun (Docker Compose)

RabbitMQ

5672, exchange tabaani.events

Une base par service

Gateway et sécurité

Tout appel passe par la Gateway avec un Bearer JWT

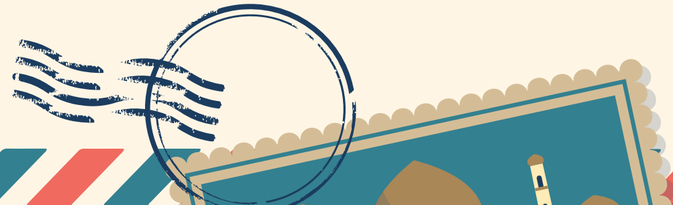
Login → Keycloak → JWT

La Gateway valide le JWT (clés publiques de Keycloak)

Les services sont dans le réseau privé, non exposés

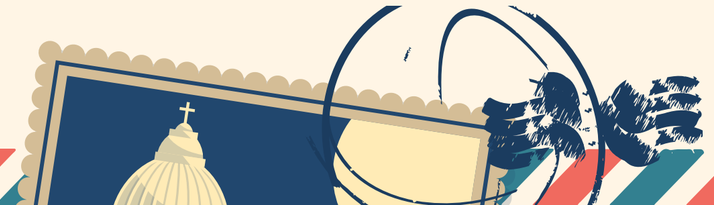
Rôles :
visiteur, utilisateur, étudiant, organisateur, enseignant, admin





Données et communications

- Une base par microservice
- Sync : OpenFeign, via Eureka (lb://)
- Async : RabbitMQ, exchange topic
- Événement = nom + clé de routage (exemple : TripPublished → voyage.publie)
- Saga : orchestrateur
- Panne : circuit breaker



Technologies



Spring Boot · Spring Cloud (Gateway, Eureka, Config Server, OpenFeign)



Keycloak · RabbitMQ · Resilience4j



PostgreSQL · MySQL · MongoDB · [Redis]



Docker Compose · Angular / React · [Flutter]

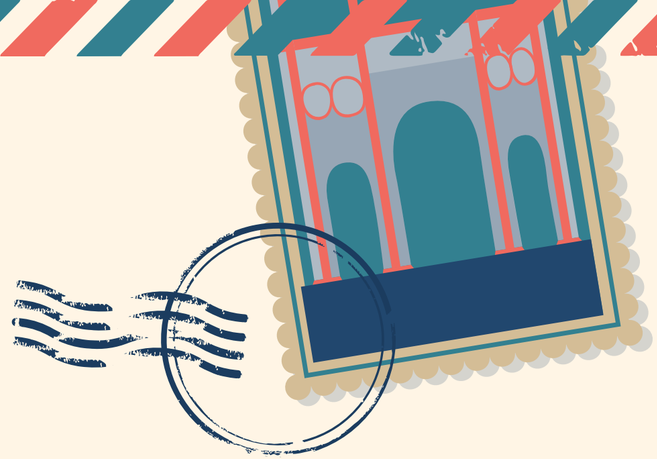


[Node.js ou Python pour le microservice avancé]



Git · Swagger





04
Conception des microservices





Nos microservices

Microservice	Rôle	Base	Techno
Utilisateurs (commun)	Identité, rôles	Keycloak	Spring Boot
Voyages et sorties	Voyages, participants	PostgreSQL	Spring Boot
Lieux et découverte	Lieux, avis	PostgreSQL	Spring Boot
Marketplace de matériel	Location de matériel	MySQL	Spring Boot
Location de voitures	Véhicules, réservations	MySQL	Spring Boot
Covoiturage	Trajets, places	PostgreSQL	Spring Boot
Paiements et forfaits	Forfaits, paiements	PostgreSQL	Spring Boot
Reclamation	Réclamations,catégories	MySQL	Spring Boot
Orchestrateur et notifs (commun)	Sagas, notifications	MongoDB	[Python / Spring Boot]

Microservice commun

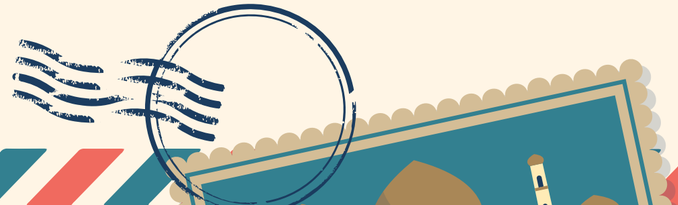
Utilisateurs

- Inscription, connexion
- Rôles, profil
- Statut étudiant

Orchestrateur et notifs

- Gestion des workflows (Saga)
- Coordination des microservices
- Gestion des événements métier
- Envoi de notifications (e-mail, alertes)
- Suivi des transactions et gestion des échecs





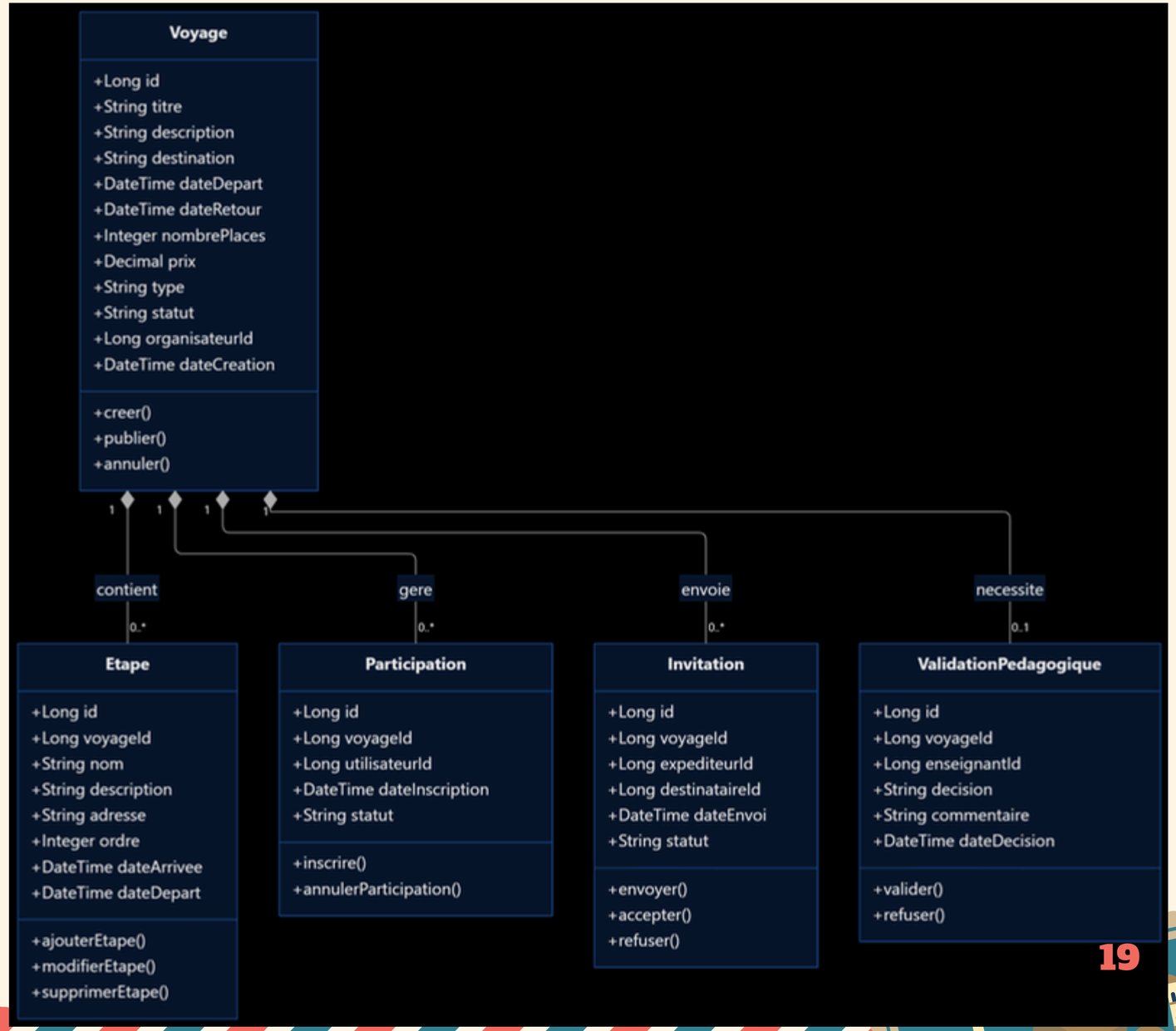
Microservice Voyages & Sorties pédagogiques

Responsabilités et fonctionalité du microservice :

- Création, modification, publication et annulation des voyages.
- Gestion des itinéraires et des étapes.
- Gestion des participants et des invitations.
- Contrôle des places disponibles.
- Validation des sorties pédagogiques par les enseignants.
- Communication avec les microservices Lieux, Utilisateurs, Paiements et Notifications.

Base de données cible :

PostgreSQL : afin de garantir la cohérence des données et la fiabilité des transactions.



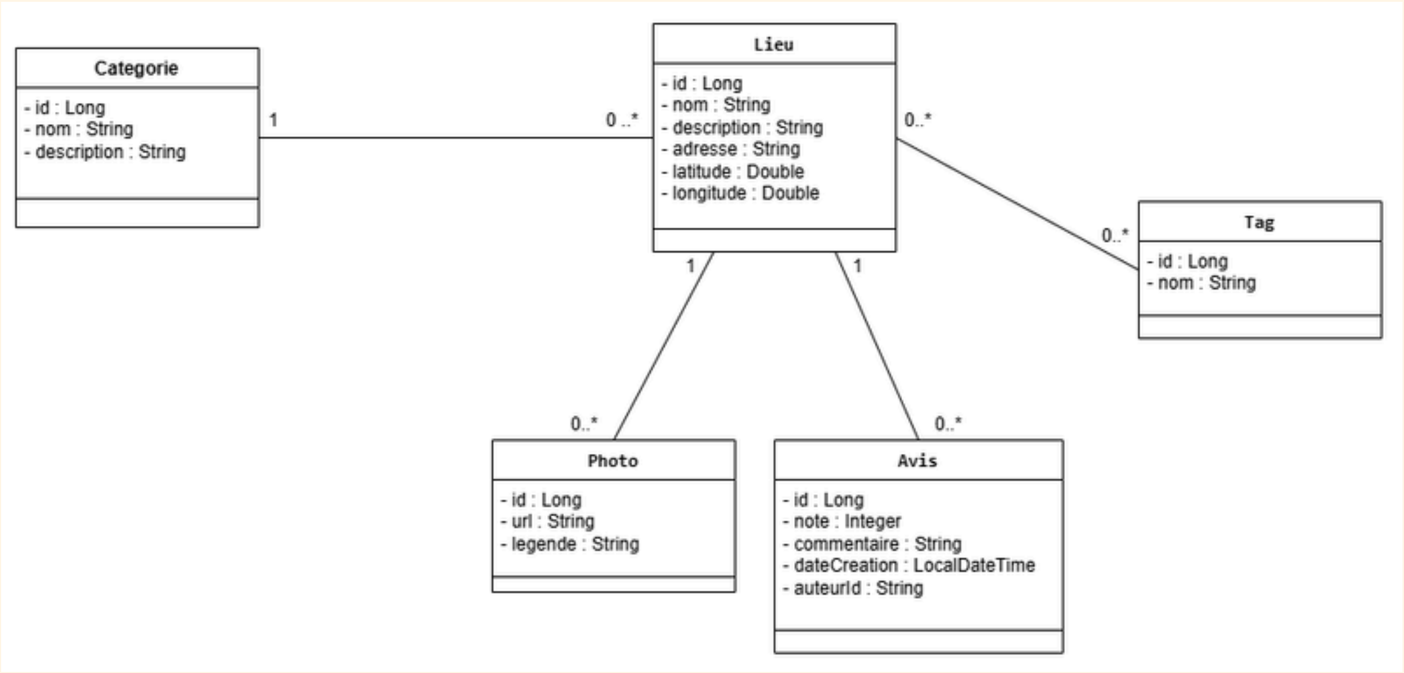
Microservice Lieux & Découverte

Responsabilités

Gérer le catalogue des lieux, leurs catégories, tags, photos et avis.

Fonctionnalités prévues

- Consulter les lieux, leurs photos et leurs avis.
- Rechercher par catégorie, tags et proximité géographique.
- Gérer les informations des lieux et publier des avis.



Base de données cible

PostgreSQL avec PostGIS

- PostgreSQL gère les données et leurs relations.
- PostGIS permet la recherche géographique.
- H2 sert aux premiers essais.

Lieux & Découverte — Interactions et API

Interactions prévues

Synchrone · OpenFeign

- **Voyages vérifie le lieu avant de l'ajouter à l'itinéraire.**
- **Service indisponible : ajout reporté.**

Asynchrone · RabbitMQ

- **LieuSupprime { lieuId } signale la suppression.**
- **Voyages marque l'étape « à revoir ».**
- **En cas de panne : message conservé et réessayé sans doublon.**

Principaux endpoints

Endpoint	Fonction	Codes
GET /api/lieux	Lister et rechercher	200, 400
GET /api/lieux/{id}	Consulter un lieu	200, 404
POST /api/lieux	Ajouter un lieu	201, 400, 401, 403
POST /api/lieux/{id}/avis	Publier un avis	201, 400, 401, 404

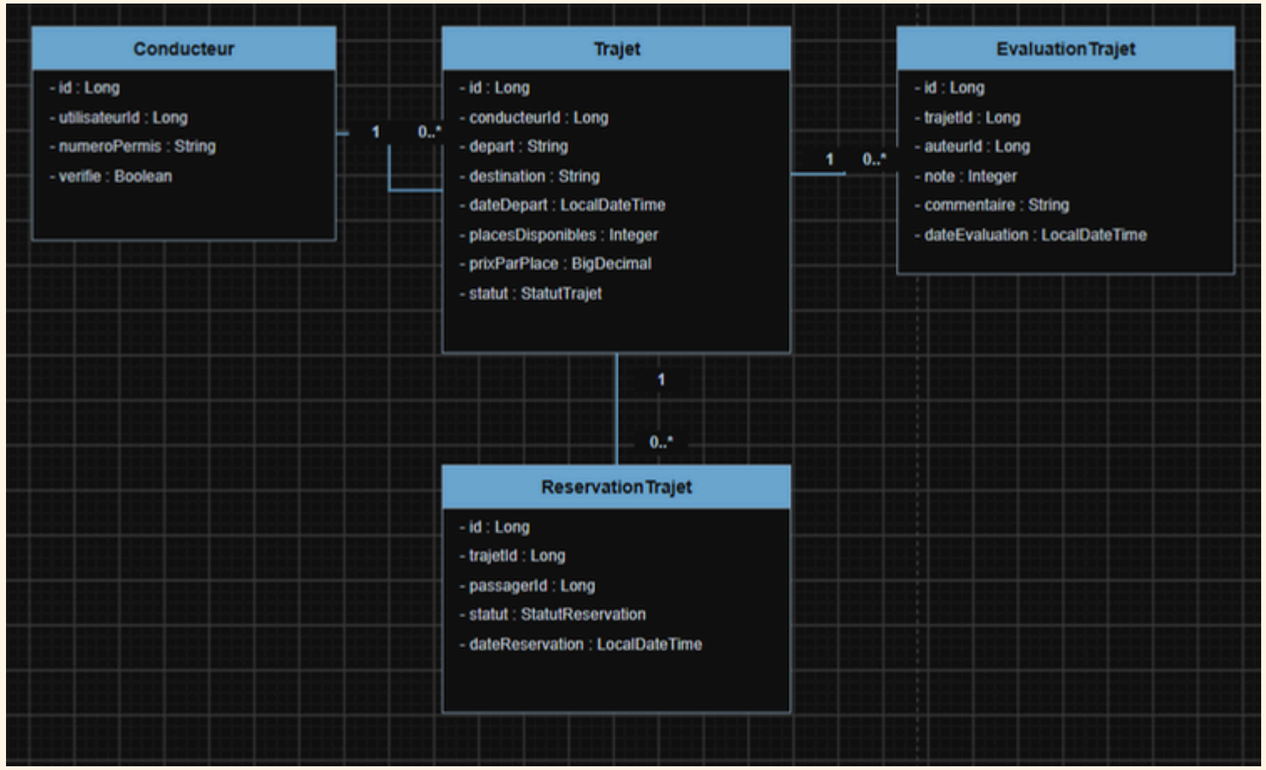
Microservice Covoiturage

Responsabilités

- Gérer les conducteurs, les trajets, les réservations de places et les évaluations des passagers.

Fonctionnalités prévues

- Proposer et rechercher des trajets selon le départ, la destination et la date.
- Réserver ou annuler une place et consulter les disponibilités.
- Gérer les profils conducteurs et évaluer les trajets effectués.



Base de données cible

PostgreSQL pour stocker les conducteurs, les trajets, les réservations et les évaluations, avec une base indépendante pour le microservice.

Covoiturage — Interactions et API

Interactions prévues

Synchrone · OpenFeign

- Covoiturage vérifie l'existence du conducteur ou du passager auprès du service Utilisateurs.
- Service indisponible : réservation reportée, sans confirmer de place.

Asynchrone · RabbitMQ

- ReservationConfirmee signale qu'une place a été réservée.
- Le service de notifications peut envoyer une confirmation aux étudiants.
- En cas de panne : événement conservé et réessayé, avec protection contre les doublons.

Principaux endpoints

Endpoint	Fonction	Codes
GET /api/trajets	Rechercher des trajets	200, 400
GET /api/trajets/{id}	Consulter un trajet	200, 404
POST /api/trajets	Proposer un trajet	201, 400, 401
POST /api/reservations	Réserver une place	201, 400, 409

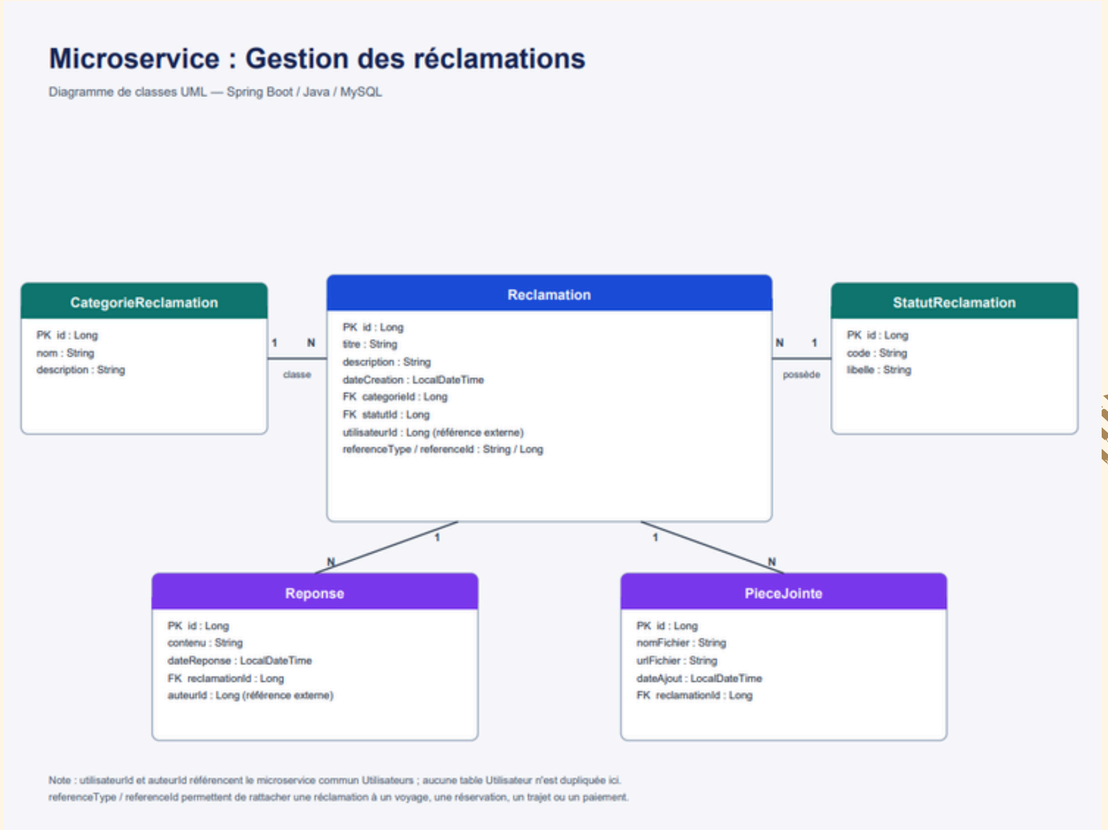
Microservice Reclamation

Responsabilités :

- Création et gestion des réclamations des utilisateurs.
- Traitement des réclamations et gestion des réponses.
- Suivi des statuts et de l'évolution des réclamations.

Fonctionnalités prévues

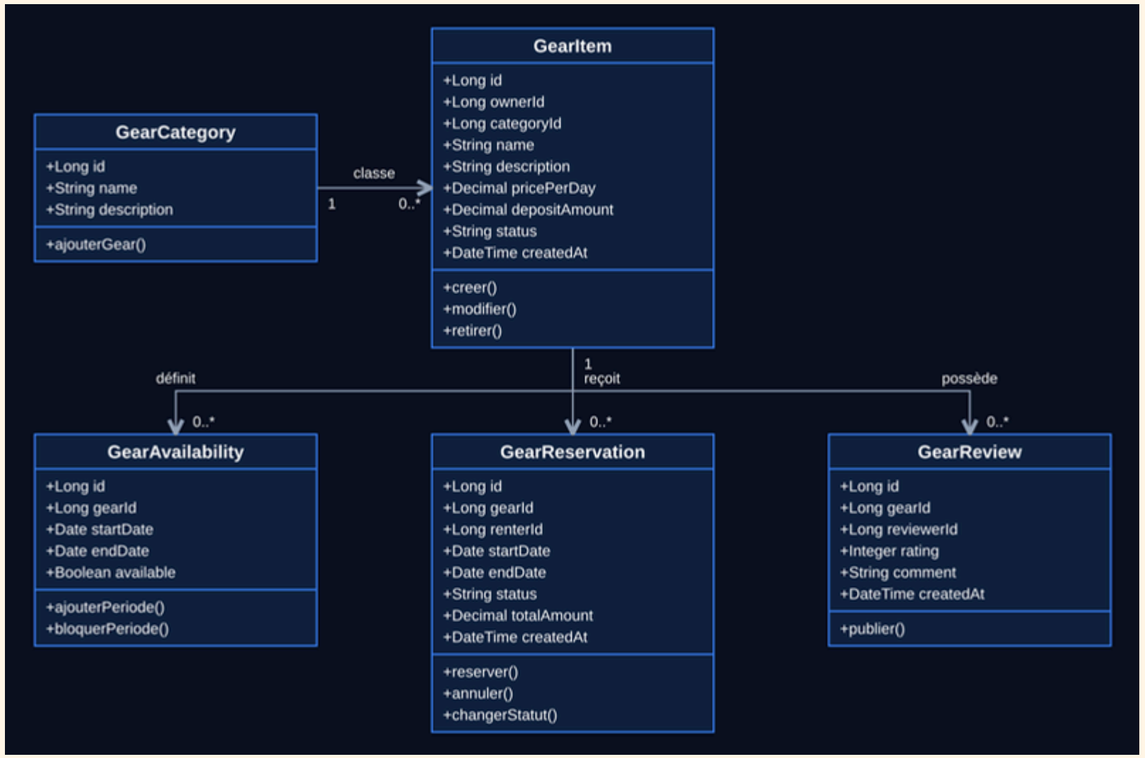
- Création de réclamations.
- Traitement des réclamations.
- Suivi des statuts.



Microservice Marketplace de materiel

Responsabilités

Gérer le catalogue du matériel proposé à la location, ses disponibilités, les réservations et les avis.



Fonctionnalités prévues

- Créer, modifier et retirer une annonce de matériel.
- Rechercher par catégorie, mot-clé et disponibilité.
- Réserver du matériel (caution traitée par Payments) et publier un avis après la location.

Base de données cible

MySQL (marketplace_db)
Données relationnelles : catégories, matériels, disponibilités, réservations et avis.

Marketplace de matériel — Interactions et API

Interactions prévues

Synchrone · OpenFeign

- **Marketplace vérifie le membre auprès de Users avant de créer une réservation.**
- **Service indisponible : réponse 503, réservation refusée.**

Asynchrone · RabbitMQ

- **GearDepositRequested { reservationId, amount } demande la caution à Payments.**
- **Payments répond GearDepositSucceeded ou GearDepositFailed.**
- **En cas de panne : message conservé et réessayé sans doublon.**

Principaux endpoints

Endpoint	Fonction	Codes
GET /api/gears	Lister et rechercher	200
GET /api/gears/{id}	Consulter un matériel	200, 404
POST /api/gears	Ajouter une annonce	201, 400, 401, 403
POST /api/reservations	Demander une réservation	202, 400, 401, 403, 404, 409, 503

Communication et API REST de Reclamation

Méthode	URI	Codes
POST	/api/reclamations	201
GET	/api/reclamations	200 / 404
GET	/api/reclamations/	200 / 404
PUT	/api/reclamations/	200
PATCH	/api/reclamations//statut	200
POST	/api/reclamations//reponses	201

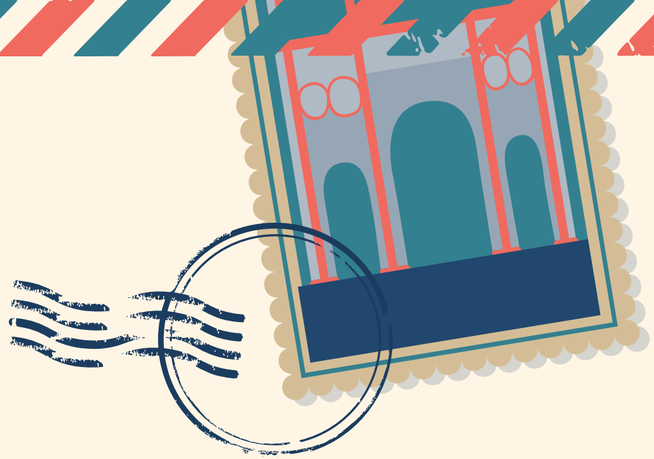
Diagramme d'interactions (exemple)

Méthode	URI	Codes
POST	/api/trips	201, 400
GET	/api/trips/{id}	200, 404
PATCH	/api/trips/{id}/status	200, 409
POST	/api/trips/{id}/participants	201, 409
DELETE	/api/trips/{id}/participants/{userId}	204, 404
POST	/api/trips/{id}/invitations	201, 403
PATCH	Forfaits, paiements	200, 403

05

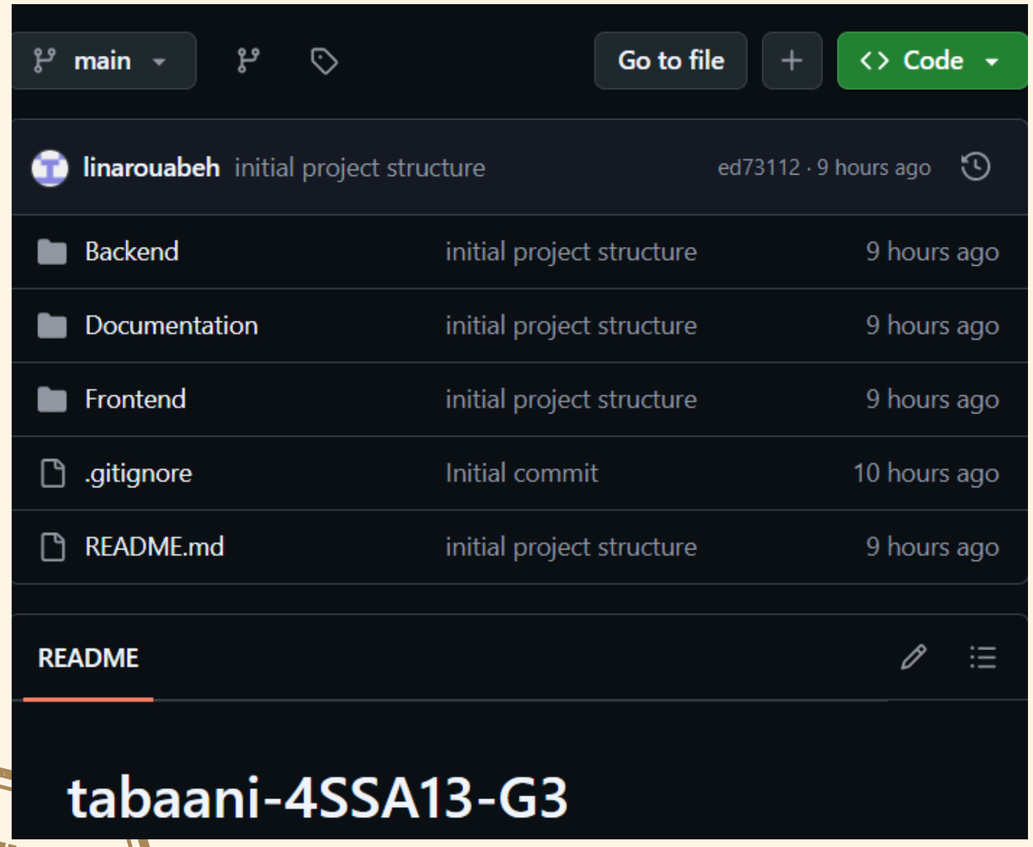
Organisation et gestion du projet

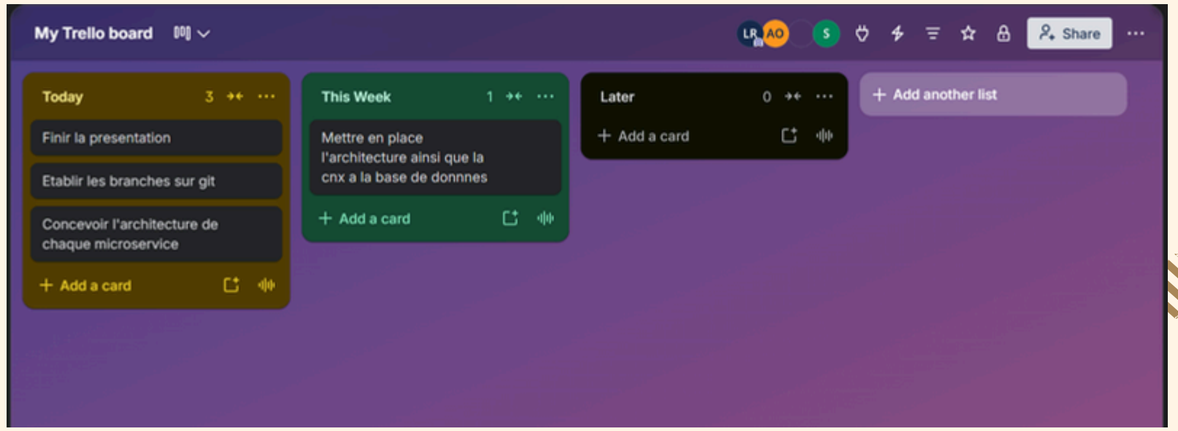




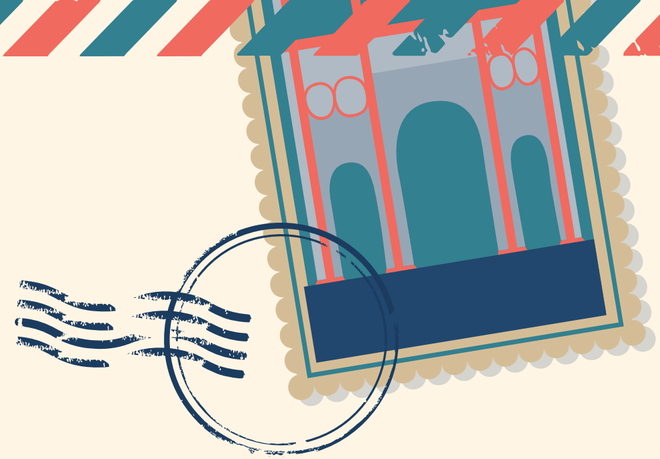
Répartition des tâches

Git + suivi









06

Planification et perspectives





Planning prévisionnel

01	**Validation intermédiaire :** Valider la conception, l'architecture et la répartition des tâches	
02	**Développement des services** Implémenter les fonctionnalités prioritaires et les API	
03	**Intégration** Connecter le frontend, les services, la sécurité et les communications	
04	**Tests et corrections** Vérifier les parcours utilisateurs, les échanges et les cas de panne	
05	**Préparation du rendu final** Finaliser la documentation, le déploiement Docker et la démonstration	

Prancipaux risques

1 Périmètre trop ambitieux

Prioriser les fonctionnalités essentielles.

2 Incompatibilité entre services

Définir des contrats d’API et des versions techniques communes.

3 Retard d’intégration

Intégrer progressivement et suivre les tâches chaque semaine.

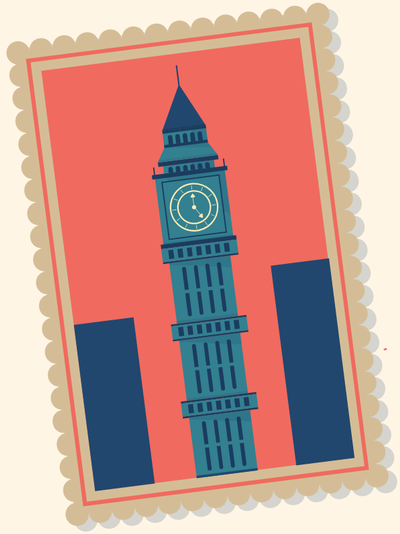
4 Échec de réservation ou de paiement

Prévoir des actions de compensation dans l’orchestrateur.

Perspectives

- Enrichir le catalogue de lieux et les offres partenaires.
- Proposer des recommandations adaptées aux préférences des voyageurs.
- Améliorer l’expérience mobile et le suivi du convoi en direct





Tabaani

Merci pour votre attention !!

Nous sommes à votre écoute pour vos questions

